# Overview and Part 1: Bezier curves

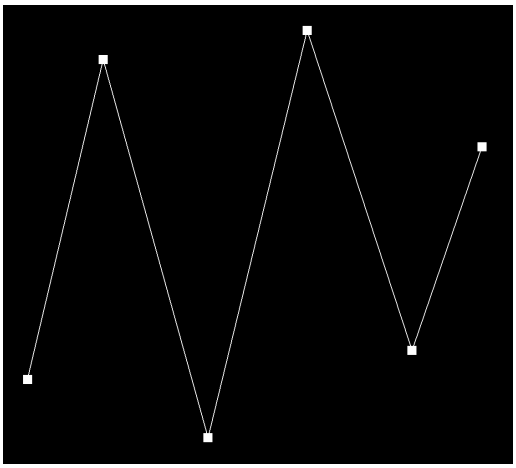
Weishu Zhang | CS184 Fall 2026

Webpage: cal-cs184-student.github.io/hw2-tree-writeup/
Private code repository: github.com/cal-cs184-student/hw2-tree

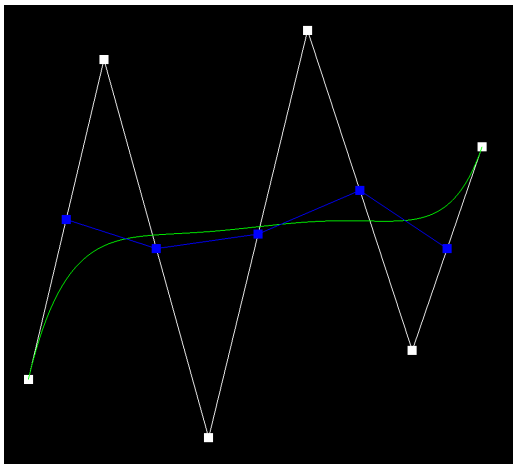
This project implements Bezier curves and patches, area-weighted vertex normals, local edge flips and splits, and Loop subdivision. The same local interpolation idea evaluates a curve or a surface; mesh operations additionally require preserving connectivity. Separating topology changes from position updates makes subdivision predictable and avoids using partially updated geometry.

de Casteljau evaluation repeatedly linearly interpolates neighboring points: p' = (1-t)p_i + t p_(i+1). evaluateStep returns exactly one level, so six original control points produce levels with 5, 4, 3, 2, and 1 point. A degree-five curve requires five steps. The GUI screenshots below were captured by pressing E; C draws the completed curve.

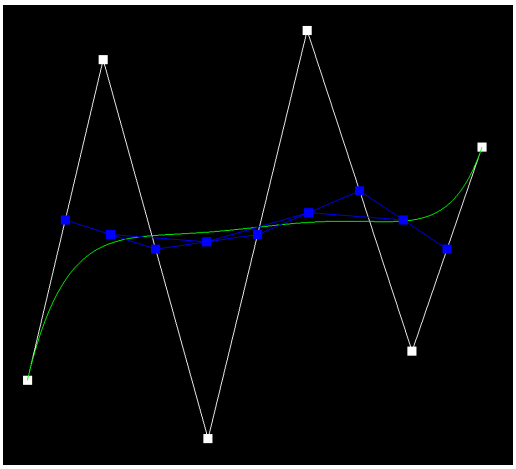
The custom control points in bzc/custom6.bzc are (0.12,0.25), (0.25,0.80), (0.43,0.15), (0.60,0.85), (0.78,0.30), and (0.90,0.65). The initial evaluation parameter is t=0.5. White points form the control polygon, blue points are intermediate interpolants, and the final point is red.



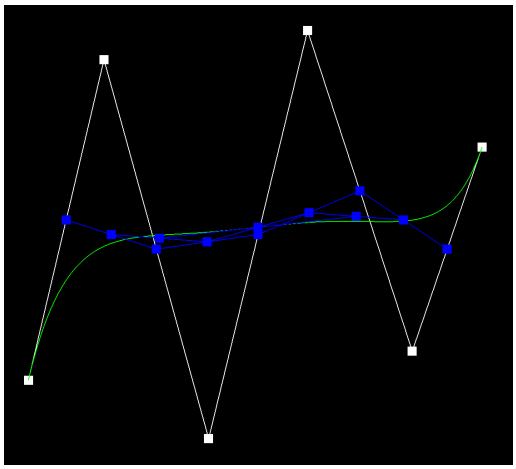
Level 0: 6 points in the newest level, t=0.5



Level 1: 5 points in the newest level, t=0.5



Level 2: 4 points in the newest level, t=0.5

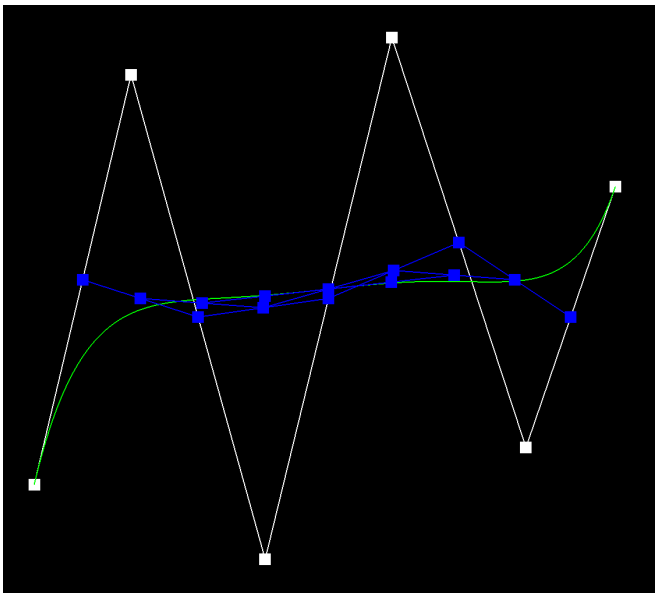


Level 3: 3 points in the newest level, t=0.5

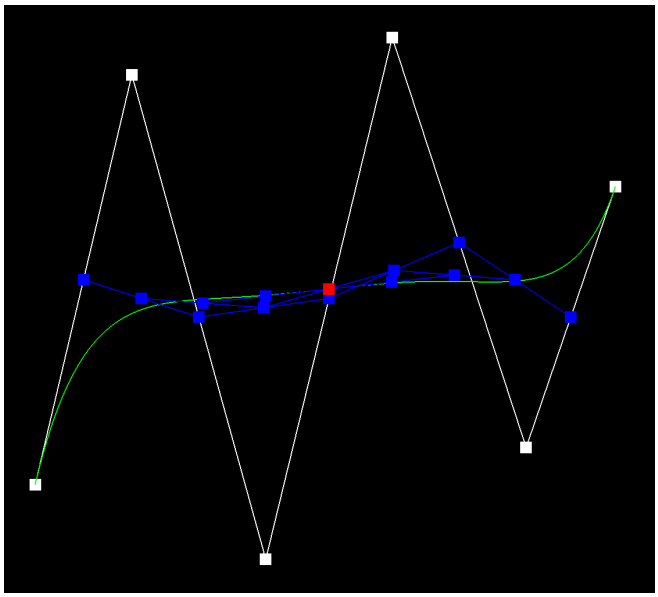
# Part 1: Final evaluation and interactive edits

The fifth level contains one point on the green curve. At t=0.5, the computed point is (0.5146875,0.5125); the blue levels converge to this red point. At t=0 and t=1, repeated interpolation returns the first and last control points respectively.

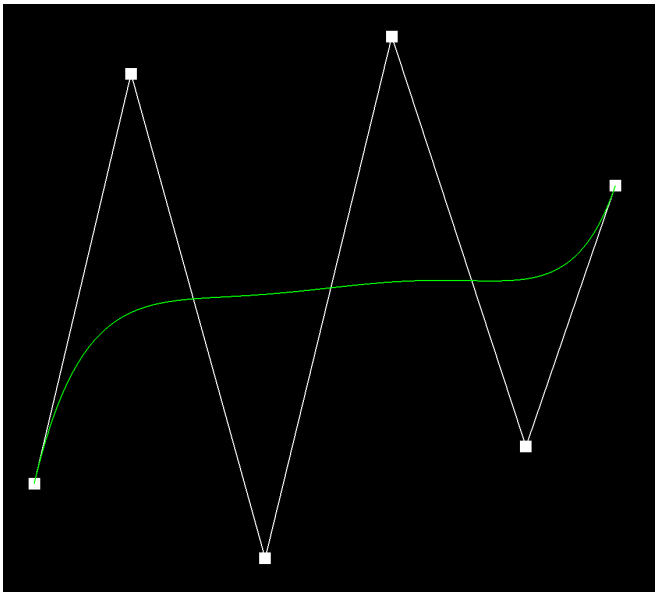
The final screenshot changes the second control point from (0.25,0.80) to (0.52,0.55), using bzc/custom6-adjusted.bzc, then changes t by scrolling in the viewer. The green curve responds to the changed control polygon, while the red evaluation point moves along the new curve. This distinguishes changing the curve itself from selecting another point on the same curve.



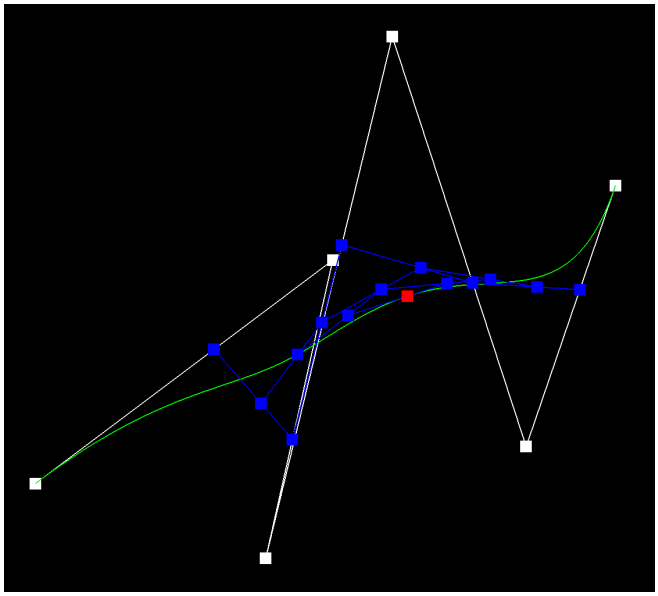
Level 4: two points remain



Level 5: final point on the curve



Completed degree-five curve



Modified control polygon and parameter

# Part 2: Bezier surfaces

The patch evaluation uses separable one-dimensional de Casteljau evaluation. evaluateStep interpolates neighboring Vector3D values. evaluate1D repeatedly calls this helper until one point remains. evaluate(u,v) first evaluates every row of the control-point grid at u, then evaluates the resulting list of row points at v.

This performs tensor-product interpolation without treating the surface as a single flattened curve. The screenshot shows bez/teapot.bez evaluated by this implementation, then tessellated by the supplied patch-to-mesh code. The body, lid, handle, and spout are all reconstructed from Bezier patches.

A numerical check uses a bilinear patch with corners (0,0,0), (1,0,1), (0,1,1), and (1,1,2). Evaluating at (u,v)=(0.25,0.75) returns (0.25,0.75,1), as expected. The implementation also handles a single-point curve and returns a zero vector for an empty input.



Bezier patch tea pot: bez/teapot.bez

# Part 3: Area-weighted vertex normals

For each vertex, normal() walks its incident halfedges with h = h->twin()->next(). For each real incident triangle, it accumulates cross(p_1-p_0, p_2-p_0). The magnitude of this cross product is twice the triangle area, so this sum already weights each face normal by area. Normalizing the sum produces the vertex normal.

Virtual boundary faces are excluded because they are not part of the surface. A zero-length sum returns the zero vector instead of dividing by zero. The traversal uses const iterators and does not modify the mesh.

The images use dae/teapot.dae with identical geometry and camera. Q switches between flat face normals and the computed vertex normals. Flat shading exposes bands and facets; vertex-normal interpolation gives continuous lighting across the body and handle. The silhouette stays polygonal because shading does not add geometry.



Flat shading, Q off



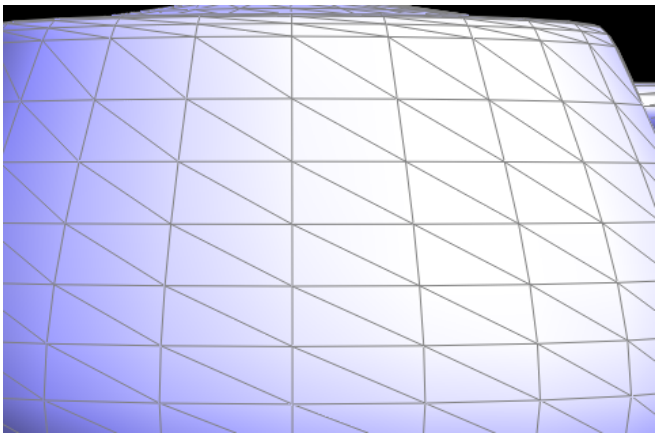
Area-weighted vertex normals, Q on

# Part 4: Edge flip

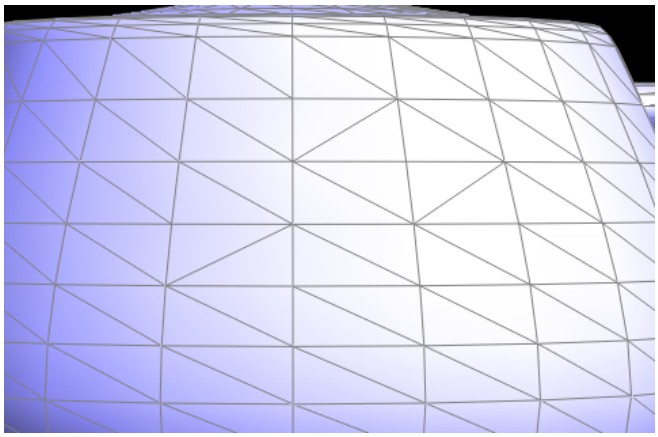
The two triangles adjacent to an interior edge are collected before any pointer changes. With triangles (a,b,c) and (b,a,d), the shared a-b edge becomes c-d, giving triangles (c,d,b) and (d,c,a). The operation reuses the same six halfedges, two faces, and edge; it changes no vertex positions and allocates nothing.

setNeighbors sets next, twin, vertex, edge, and face together. After rewiring both triangle cycles, the representative halfedge pointers of all four vertices, both faces, and the affected edges are reset. Boundary edges return unchanged. The interior operation does constant work.

The close-ups show three flips on neighboring diagonals of the teapot body. The changed diagonal directions are visible, while the surrounding surface remains connected. Verification checks twin reciprocity, face cycles, endpoint consistency, representative pointers, and unchanged element counts. Flipping the same test edge twice also restores its original diagonal.



Teapot body before flips



Teapot body after three local flips

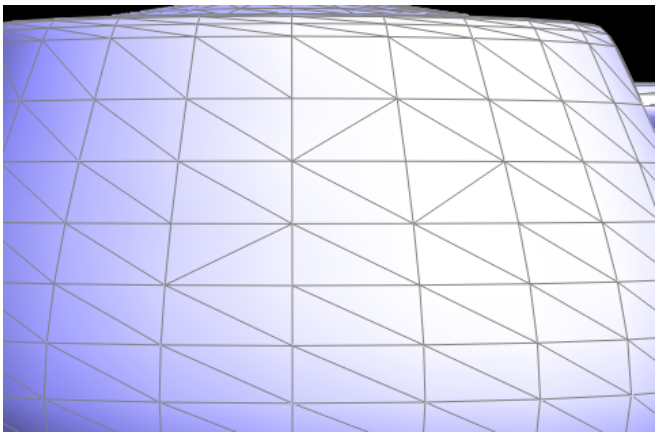
# Part 5: Edge split and mixed operations

An interior split adds the midpoint m=(a+b)/2 and replaces the adjacent pair of triangles with four triangles: (a,m,c), (m,b,c), (m,a,d), and (b,m,d). It adds one vertex, three edges, two faces, and six halfedges. The outer edges and their outside twins are retained.

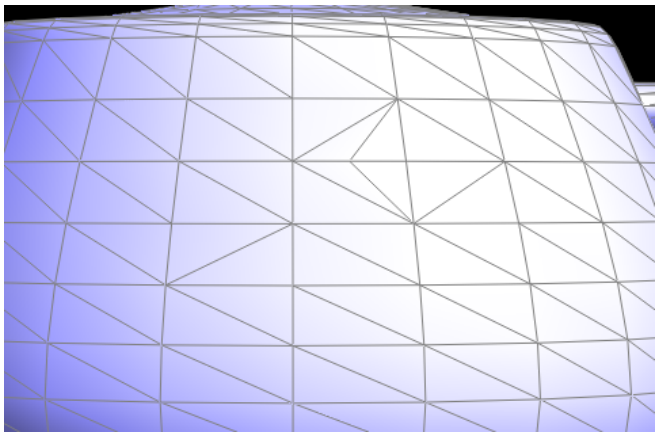
The new vertex points along the original edge (m->a), as required. The two child segments of the old edge are marked isNew=false; the two spokes to the opposite vertices are marked true. This distinction is needed when Loop subdivision later decides which new edges to flip.

The first row isolates one split, performed after the flips in Part 4. The second row compares the original region with the combined flip-and-split result. The midpoint and extra spokes appear without holes. The topology test also performs 80 mixed operations on a subdivided tetrahedron and validates connectivity after every operation.

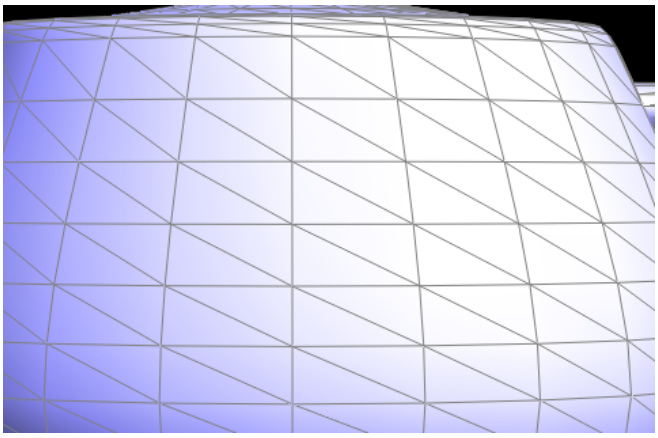
The initial test harness uncovered a supplied build() detail: virtual boundary faces do not initially receive a representative halfedge. The checker was adjusted to validate the boundary halfedge loops directly. This was a fixture assumption, not a failure of the local split.



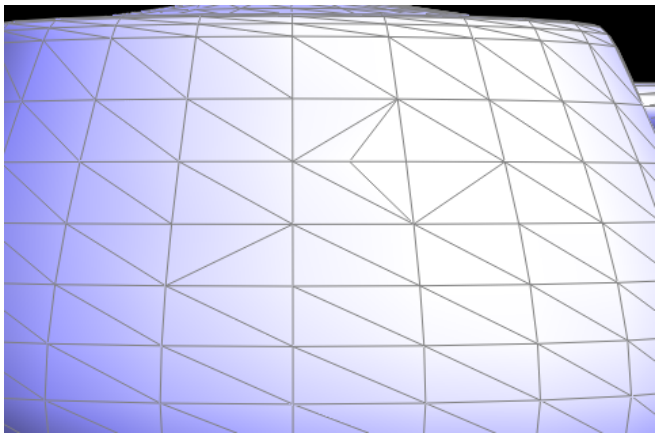
Before one interior split



After one interior split



Before the mixed flip/split sequence



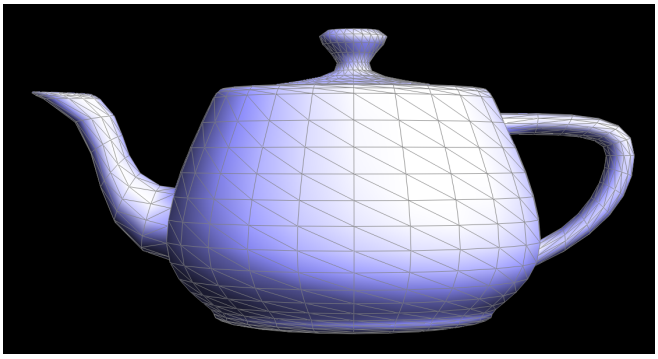
After three flips and one split

# Part 6: Loop subdivision

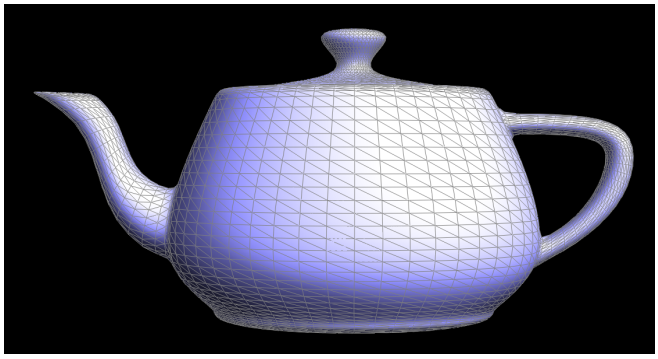
All new positions are computed from the original geometry before editing topology. For an old interior vertex of valence n, p'=(1-n beta)p + beta sum(neighbors), with beta=3/16 for n=3 and beta=3/(8n) otherwise. For an interior edge (a,b) with opposite vertices (c,d), its new point is 3/8(a+b)+1/8(c+d).

Original edges are saved in a vector of stable list iterators. Only these edges are split; iterating to the changing end of the list would split freshly created edges indefinitely. Each inserted vertex receives the position cached on its original edge. Then new spokes joining an old and a new vertex are flipped. Finally every vertex copies newPosition into position.

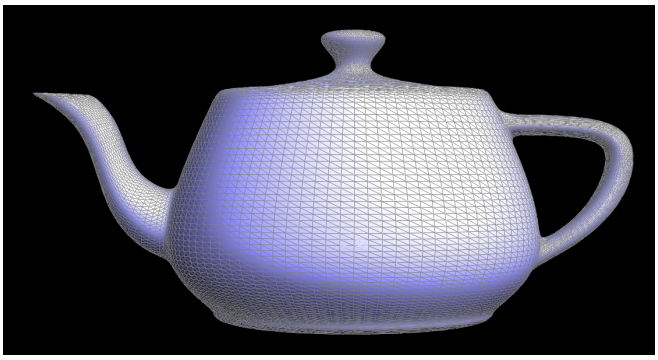
At the beginning of each round, all current vertices and edges become old again. This is essential for repeated subdivision. The tea pot gains four times as many triangles per round, while the handle, lid knob, and spout become smoother. Fine details and sharp changes gradually round off because the position rules average neighboring points.



Original dae/teapot.dae



One Loop subdivision



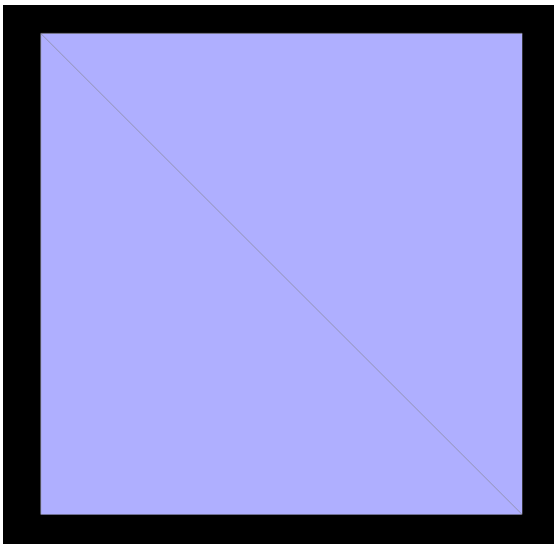
Two Loop subdivisions

# Part 6: Cube asymmetry

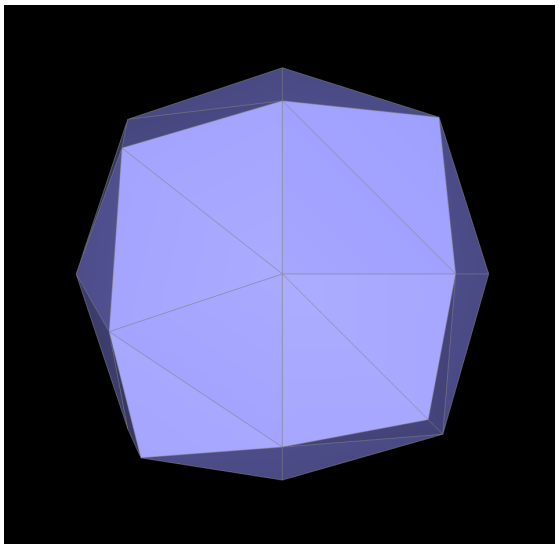
The original cube has one diagonal per square face. These diagonal choices are not invariant under all cube reflections: some corners have different valences and different one-ring neighbors. Loop weights therefore move corners in different directions even though the original positions form an approximately symmetric cube.

The screenshots use dae/cube.dae from the provided assets, at the same front-facing camera for all rounds. The first round already creates a tilted boundary; subsequent rounds round the corners and preserve a directional bias. Increasing resolution alone does not repair the original asymmetric stencil.

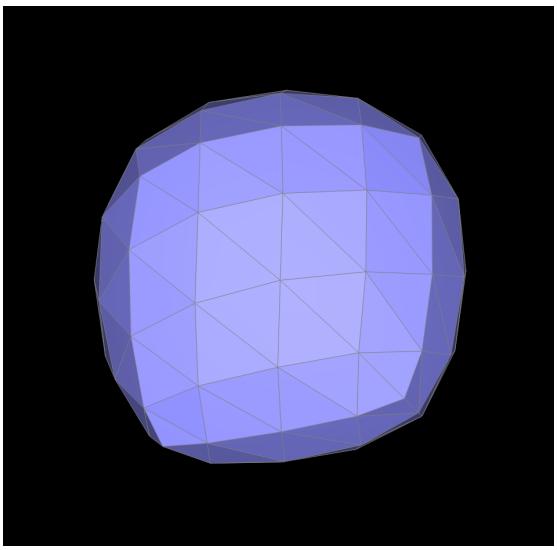
A separate numerical experiment rounds the supplied near-integer cube coordinates to exact +/-1 and measures the maximum distance from a reflected vertex to its nearest mesh vertex, across the three coordinate-plane reflections. The original mesh has errors 0, 0.414578, 0.234375, and 0.191886 after rounds 0 through 3. This is a geometric symmetry measure, not a grading score.



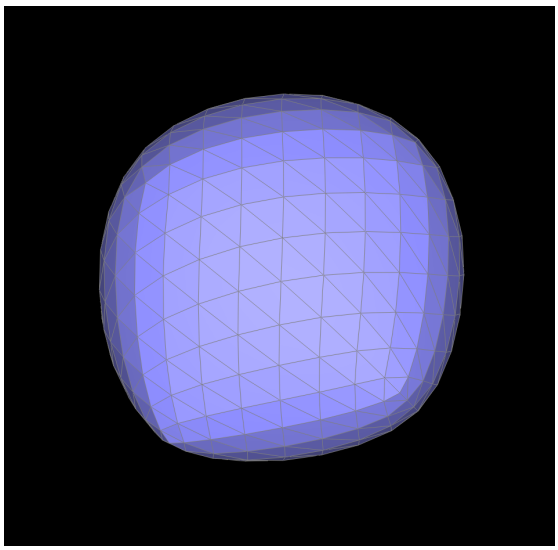
Original cube: 0 Loop rounds



Original cube: 1 Loop rounds



Original cube: 2 Loop rounds



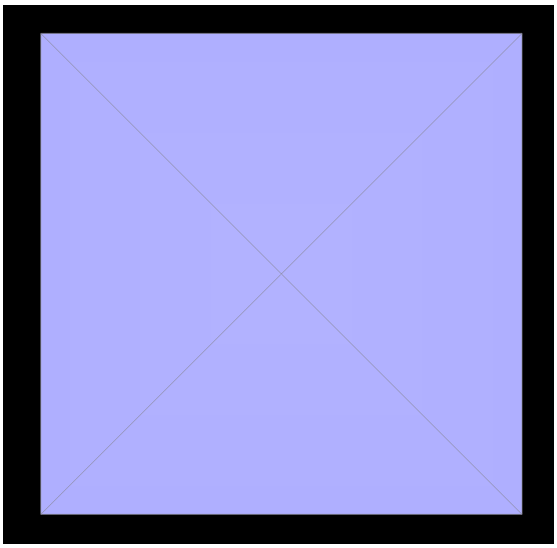
Original cube: 3 Loop rounds

# Part 6: Symmetric preprocessing and sharper features

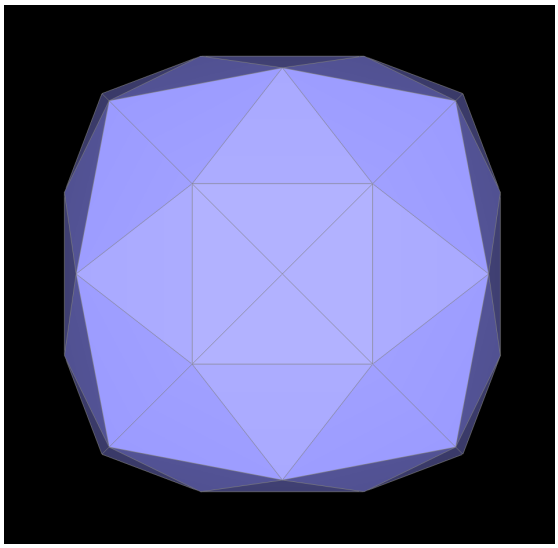
To make the cube subdivision symmetric, split each of its six coplanar face diagonals at its midpoint. Every square then has one face-center vertex and four triangles. There are 14 vertices and 24 triangles before subdivision. Each original corner has three cube-edge neighbors and three adjacent face-center neighbors, so all corners have equivalent valence-six stencils.

This preprocessing is produced by the implemented splitEdge function, rather than manually drawing a smooth replacement. The exported cube-symmetric.dae has exact +/-1 original coordinates to remove the tiny asymmetries in the asset. The original and processed meshes have the same cube shape before smoothing.

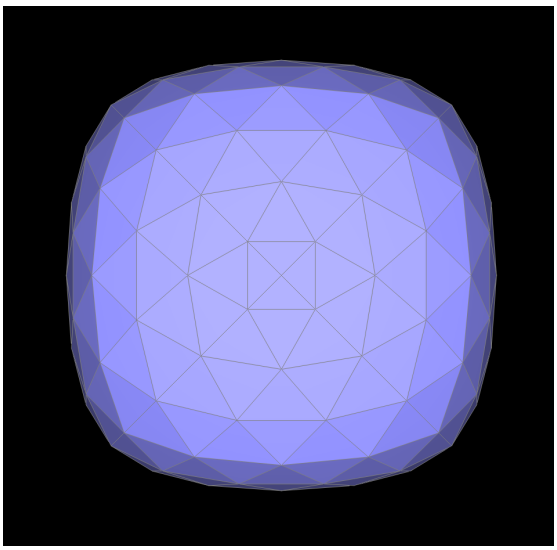
All three coordinate-plane reflection errors remain zero through three rounds in the exact-coordinate experiment. The screenshots show a balanced rounded square. The mesh still rounds off: symmetry fixes inconsistent neighborhoods, while preserving a crease would require additional crease rules or supporting geometry.



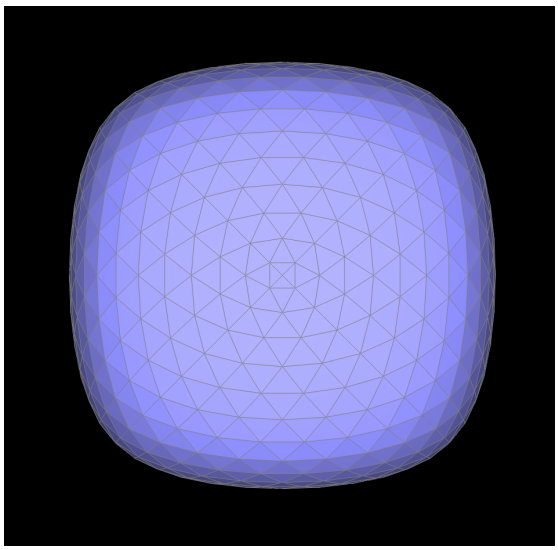
Face-center preprocessing: 0 Loop rounds



Face-center preprocessing: 1 Loop rounds



Face-center preprocessing: 2 Loop rounds



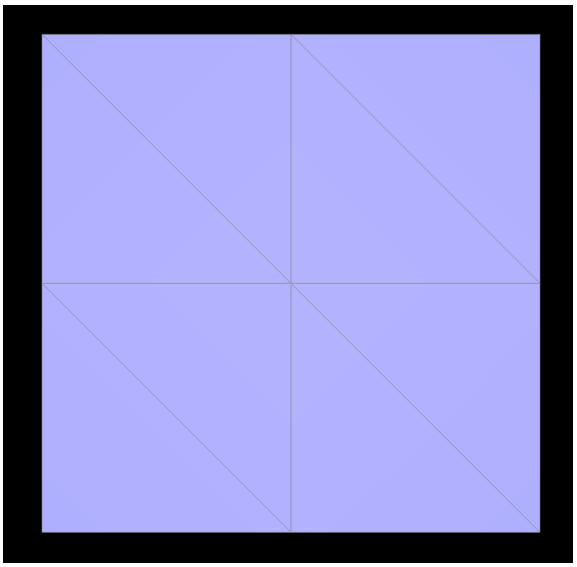
Face-center preprocessing: 3 Loop rounds

# Part 6: Reducing corner rounding with pre-splits

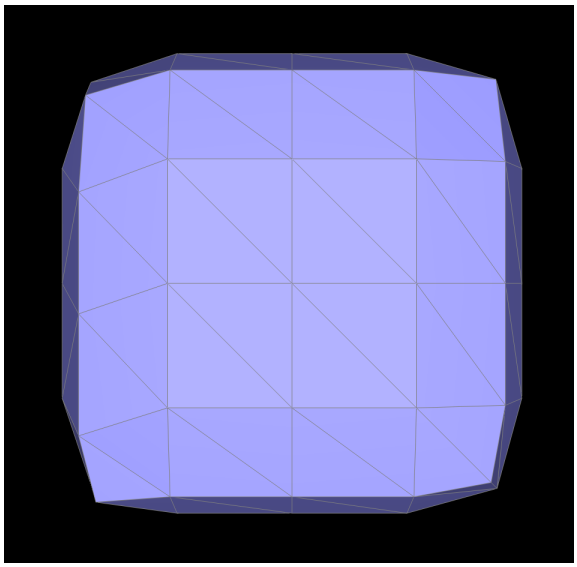
An additional experiment applies one linear 4-to-1 refinement before Loop subdivision: split every original edge at its geometric midpoint, then flip the new old-to-new spokes, without applying the smoothing position rules. This adds samples while keeping the starting cube surface unchanged.

At the same number of subsequent Loop rounds, this denser starting mesh has smaller neighborhoods near each corner and retains a squarer outline. Pre-splitting reduces the amount of rounding per finite round; it does not permanently preserve an infinitely sharp crease. It also does not correct the original diagonal bias, unlike the face-center preprocessing.

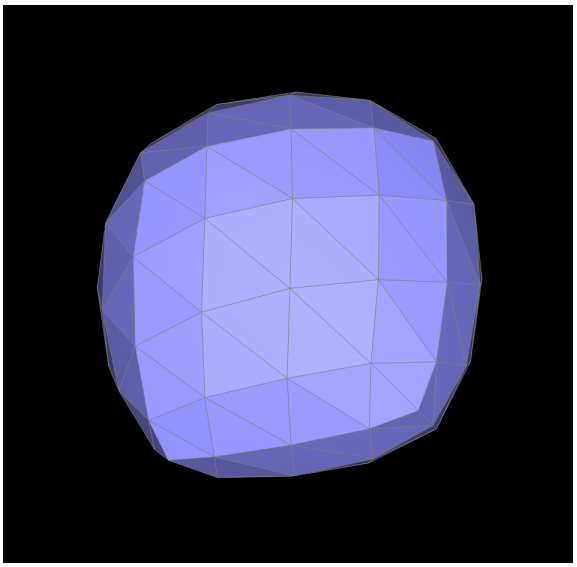
The lower row compares two Loop rounds directly. Both views use the same camera and original extent. The presplit cube keeps broad flat regions and tighter corners, while the coarse original cube has shrunk and rounded more.



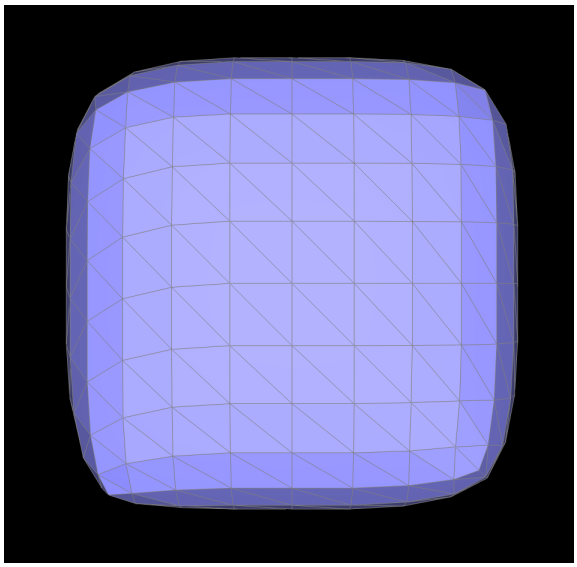
Linear refinement before smoothing



Presplit cube after one Loop round



Coarse cube after two Loop rounds



Presplit cube after two Loop rounds

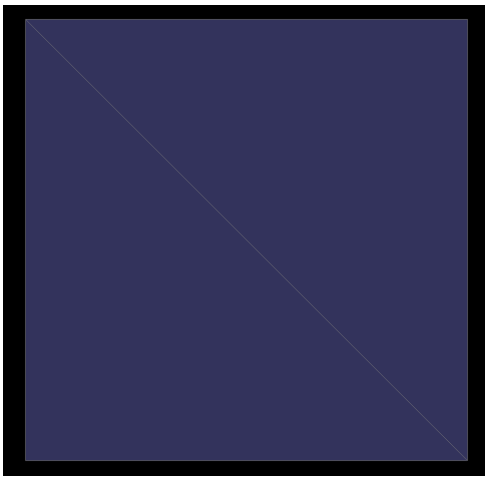
# Extra credit: Boundary splits and boundary subdivision

Boundary splitting is supported. The halfedge is oriented into the real face; only that triangle is divided. The virtual boundary loop replaces b->a with b->m->a, and the new midpoint is connected to the real triangle's opposite vertex. This adds one vertex, two edges, one real face, and four halfedges. Boundary flips remain disabled.

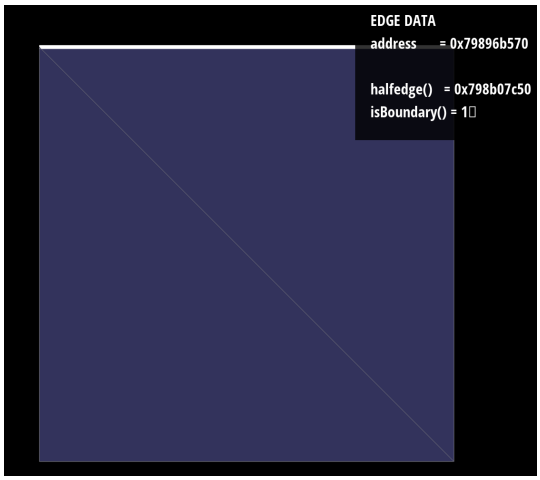
Loop subdivision uses the boundary edge midpoint (a+b)/2 and the old boundary-vertex rule p'=3/4 p + 1/8(left+right). Normals ignore the virtual boundary face. The screenshots show a square with two triangles, a selected edge with isBoundary()=1, the result of S on that edge, and the result of L on the open mesh.

Automated verification covers numerical Bezier values, planar normals, reciprocal twins, triangle cycles, matching endpoints, representative vertex/edge/real-face pointers, element counts, boundary midpoint placement, four-fold face growth, three open-mesh subdivision rounds, four closed-mesh rounds, and 80 mixed local operations. AddressSanitizer and UndefinedBehaviorSanitizer complete without errors.

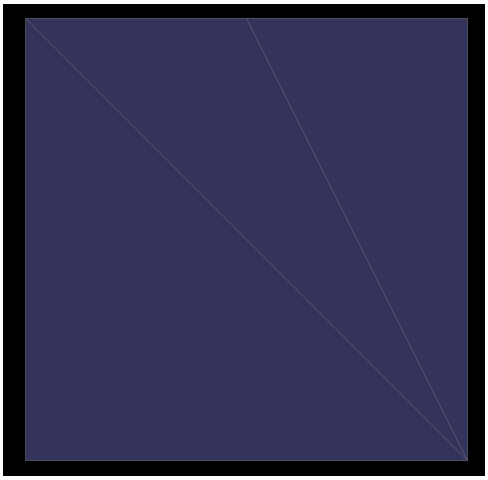
Tool assistance: Codex assisted with implementation, automated checks, computer-use experiments, and report drafting. Figures were captured from the actual renderer framebuffer in a local preview build to exclude cursor overlays. No art-competition model or additional subdivision scheme is claimed.



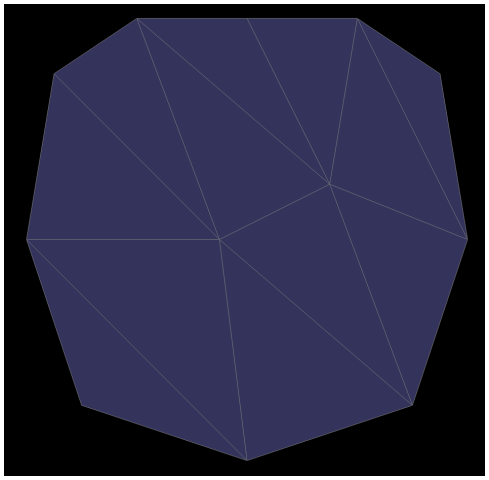
Open square before editing




Selected edge: isBoundary() = 1



After splitting the top boundary edge



After one boundary-aware Loop round